Neural Networks 206 (2027) 109799

Contents lists available at ScienceDirect

Neural Networks

journal homepage: www.elsevier.com/locate/neunet

Full Length Article

Moving beyond graph homophily: Disentangled node representation for graph neural networks

Shuyang Fang [a], Zelong Yang [b], Yintao Cai [a], Yuqin Huang [a], Xiaoping Min [a,*]

[a] School of Informatics, State Key Laboratory of Molecular Vaccinology and Molecular Diagnostics, National Institute of Diagnostics and Vaccine Development in Infectious Diseases, Xiamen University, Xiamen, Fujian, 361005, China
[b] Institute of Artificial Intelligence, Xiamen University, Xiamen, Fujian, 361005, China

ARTICLE INFO

Keywords:
Graph representation learning
Graph neural networks
Heterophily
Representation disentanglement
Node classification

ABSTRACT

Graph Neural Networks (GNNs) have demonstrated strong performance on graph-structured data through neighborhood aggregation. However, they often underperform on heterophilic graphs and suffer from over-smoothing, leading to suboptimal representations. We attribute these limitations to the lack of view-specific factorization in current node representations, which fail to reflect the multifaceted nature of node features. In this paper, we propose a novel node representation paradigm that decomposes node features into three complementary perspectives: label context, semantic, and topology. We observe that the relative importance of these perspectives varies not only across datasets but also among nodes within a single graph. To address this, we introduce RDGCN (Representation-Disentangled Graph Convolutional Network), which employs view-specific encoders and an adaptive fusion mechanism to learn and integrate multi-view representations dynamically. RDGCN mitigates harmful information mixing through view-specific factorization and enhances the discriminability of each encoder via targeted regularization. Notably, our framework is based on simple MLP and GNN modules yet achieves strong performance, demonstrating the power of multi-view representation learning. Moreover, our work establishes a generalizable node representation paradigm and offers a unified framework that moves beyond the binary homophily/heterophily divide. Extensive experiments on nine benchmark datasets validate the effectiveness of RDGCN, showcasing robust performance across both homophilic and heterophilic graphs.

1. Introduction

Graph Neural Networks (GNNs) (Bruna et al., 2014; Defferrard et al., 2016; D'Inverno et al., 2025; Guo et al., 2025; Hamilton et al., 2017; Kipf & Welling, 2017; Lin et al., 2025; Scarselli et al., 2008; Veličković et al., 2018) are deep learning models specifically designed for graph-structured data. They have been widely applied in domains such as social networks (Li & Goldwasser, 2019), biomedical information networks (Yan et al., 2019), and citation networks (Zhao et al., 2021). In particular, GCN (Kipf & Welling, 2017) and its variants (Hamilton et al., 2017; Veličković et al., 2018; Wu et al., 2019; Xu et al., 2019), which iteratively update node representations by aggregating information from neighboring nodes based on the message passing mechanism (Gilmer et al., 2017), have demonstrated strong performance on a variety of tasks, including node classification (Rong et al., 2020; Wang et al., 2020), link prediction (You et al., 2019), and recommendation (Liu et al., 2020; Xu et al., 2018b).

However, a core mechanism of standard GNNs can be interpreted as a form of low-pass filtering (Wu et al., 2019). As more layers are stacked, the representations of a node and its neighbors tend to become increasingly similar, diminishing inter-node discriminability and leading to the over-smoothing problem (Li et al., 2018). Moreover, when applied to heterophilic graphs, where adjacent nodes are likely to belong to different classes, such aggregation can introduce noise and further degrade performance. These observations suggest that standard GNNs are generally ineffective under heterophilic scenarios, which are common in real-world graphs.

Over-smoothing can be effectively mitigated by incorporating the residual connection concept to balance original node features and neighborhood information (Chen et al., 2020; Klicpera et al., 2019; Xu et al., 2018a). Recently, increasing attention has been devoted to addressing heterophily. Some methods (Abu-El-Haija et al., 2019; Chen et al., 2023; Song et al., 2023) integrate multi-hop information to enhance performance on both homophilic and heterophilic graphs. Other methods (Jin et al., 2021; Liang et al., 2024; Pei et al., 2020) refine the message

* Corresponding author.
E-mail addresses: fangshuyang@stu.xmu.edu.cn (S. Fang), yangzelong@stu.xmu.edu.cn (Z. Yang), caiyintao@stu.xmu.edu.cn (Y. Cai), huangyuqin@stu.xmu.edu.cn (Y. Huang), mxp@xmu.edu.cn (X. Min).

https://doi.org/10.1016/j.neunet.2026.109799
Received 2 February 2026; Received in revised form 11 July 2026; Accepted 23 October 2026
Available online 26 October 2026
0893-6080/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

passing mechanism by developing more effective aggregation strategies. Additionally, several approaches (Bo et al., 2021; Chien et al., 2021; Luan et al., 2022) improve performance from a spectral perspective by capturing both high- and low-frequency graph signals.

However, existing methods typically emphasize aggregating complex beneficial information (e.g., semantic information and topological structure) and suppressing noise within a limited embedding space, without explicitly separating and leveraging multiple complementary information sources. By conducting experiments based on the disentangled graph homophily proposed in Zheng et al. (2024), we demonstrate that the relative importance of neighborhood labels, semantic features, and structural characteristics varies significantly not only across different graph datasets but also among nodes within the same dataset. Consequently, the traditional single-perspective approach to graph homophily (Abu-El-Haija et al., 2019; Gong et al., 2023; Li et al., 2023; Platonov et al., 2023a; Zhu et al., 2020), which categorizes graphs into a simple binary classification of homophily or heterophily, fails to capture the nuanced and complex characteristics inherent in real-world graphs.

To address these limitations, we propose a novel node representation paradigm by explicitly factorizing each node's representation into three distinct components: label context, semantic, and topological information. Each component captures a specific aspect of the node through view-specific encoding, thus mitigating harmful information mixing. Based on this paradigm, we introduce the **Representation-Disentangled Graph Convolutional Network (RDGCN)**. RDGCN leverages MLPs, GNNs, and carefully designed regularization strategies to construct view-specific encoders that learn representations of label context, semantic, and topological features. These view-specific representations are adaptively fused based on local graph characteristics to inform node classification. RDGCN adopts a sequential learning process consisting of view-specific factorization, specialized encoding, and importance-aware fusion. This step-wise design effectively mitigates noise caused by information mixing, thereby enhancing the robustness of node representations. Extensive experiments conducted on both homophilic and heterophilic graphs demonstrate that RDGCN consistently achieves superior performance. Our simple yet effective approach underscores the potential benefits of multi-view representation learning.

The main contributions of this work are summarized as follows:

- We conduct a comprehensive analysis of feature importance, revealing that the contributions of semantic features, neighborhood labels, and structural features vary significantly across datasets and even among nodes within the same graph.
- We propose a novel node representation paradigm that generalizes across diverse graph types, moving beyond the binary homophilic–heterophilic categorization.
- We present RDGCN, a framework built upon simple MLP and GNN modules, which generates comprehensive node representations through three view-specific encoders and an adaptive fusion mechanism. We further introduce a targeted regularization strategy that guides each encoder's learning. Importantly, the three encoders are loosely coupled by design, enabling the flexible incorporation of new feature types with minimal architectural changes.
- We evaluate RDGCN on nine benchmark datasets, encompassing both homophilic and heterophilic graphs, and consistently achieve superior performance. Extensive ablation studies and visualizations validate the effectiveness of each encoder and the adaptive fusion scheme.

2. Related works

Standard GNNs perform message passing and update node representations based on node connectivity. Representative models following this paradigm include GCN (Kipf & Welling, 2017), GraphSAGE (Hamilton et al., 2017), and GAT (Veličković et al., 2018). GraphSAGE introduces neighbor sampling to reduce computational costs, while GAT leverages attention mechanisms to adaptively aggregate messages. However, limited by simplistic message passing mechanisms, standard GNNs often suffer from over-smoothing and underperform on heterophilic graphs.

To mitigate over-smoothing, various approaches have been proposed. Residual-based methods such as APPNP (Klicpera et al., 2019) and GCNII (Chen et al., 2020) incorporate initial residual connections to preserve original node features across layers. Normalization techniques, including PairNorm (Zhao & Akoglu, 2020) and NodeNorm (Zhou et al., 2020), constrain node representations to prevent collapse. Regularization strategies such as DropEdge (Rong et al., 2020) randomly remove edges during training to slow convergence of node representations. Additionally, decoupled propagation methods (Klicpera et al., 2019; Wu et al., 2019) separate feature transformation from graph propagation, enabling deeper effective propagation without stacking more trainable layers.

To address these limitations, one prominent research direction focuses on integrating multi-hop information. For instance, JKNet (Xu et al., 2018a) uses skip connections to leverage multi-layer information, while MixHop (Abu-El-Haija et al., 2019) blends features from neighbors at different hops via neighborhood mixing. HopGNN (Chen et al., 2023) preprocesses multi-hop features and integrates them through hop interactions and fusion. Ordered GNN (Song et al., 2023) splits node embeddings into blocks aligned with hop-level neighborhoods, preserving hierarchical structures.

Beyond simply extending the neighborhood scope, another line of work aims to develop more advanced aggregation strategies. To go beyond the initial adjacency matrix, U-GCN (Jin et al., 2021) constructs multiple adjacency variants (e.g., 1-hop, 2-hop, kNN) and adaptively combines them. Geom-GCN (Pei et al., 2020) computes geometric node similarity to build multi-relational graphs with customized aggregation. From a spectral view, FAGCN (Bo et al., 2021) fuses low- and high-frequency signals, and ACM-GCN (Luan et al., 2022) uses multi-pass filters with node-level adaptive weighting.

Distinct from developing more advanced aggregation strategies, a more recent paradigm seeks to separate different sources of information entirely to mitigate interference. For instance, GNN-BC (Yang et al., 2022) explicitly decouples the representations of topology and node attributes by introducing a mutual exclusion constraint to minimize their statistical dependence. M2M-GNN (Liang et al., 2024) partitions the neighborhood of each node into distinct multisets, employing a multiset-to-multiset message passing mechanism.

3. Preliminary

3.1. Notations

We consider an undirected graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ with N nodes, where $\mathcal{V}$ denotes the set of nodes and $\mathcal{E}$ denotes the set of edges. Let $\boldsymbol{A} \in \mathbb{R}^{N\times N}$ be the adjacency matrix, where $\boldsymbol{A}_{uv} = 1$ if $e_{uv} \in \mathcal{E}$, and $\boldsymbol{A}_{uv} = 0$ otherwise. The node feature matrix is denoted by $\boldsymbol{X} \in \mathbb{R}^{N\times d}$, and the one-hot label matrix by $\boldsymbol{Y} \in \mathbb{R}^{N\times C}$, where C is the number of classes. The neighborhood of node u is defined as $\mathcal{N}_u = \{v \mid e_{uv} \in \mathcal{E}\}$.

3.2. Graph homophily metrics

Graph homophily is commonly quantified by two metrics: edge homophily (Abu-El-Haija et al., 2019; Zhu et al., 2020) and node homophily (Pei et al., 2020), defined as follows:

$$h_{\text{edge}}(\mathcal{G}, \boldsymbol{Y}) = \frac{|\{e_{uv} \mid e_{uv} \in \mathcal{E}, Y_u = Y_v\}|}{|\mathcal{E}|}, \tag{1}$$

$$h_{\text{node}}(\mathcal{G}, \boldsymbol{Y}) = \frac{1}{|\mathcal{V}|} \sum_{v\in\mathcal{V}} \frac{|\{u \in \mathcal{N}_v \mid Y_u = Y_v\}|}{|\mathcal{N}_v|}. \tag{2}$$

Edge homophily measures the proportion of edges that connect nodes sharing the same label, while node homophily evaluates the average

label consistency among each node's neighbors. Both metrics range from 0 to 1, where higher values indicate stronger homophilic tendencies. These measures are topology-based and capture label consistency along graph edges.

3.3. Disentangled graph homophily

A recent study (Zheng et al., 2024) introduces disentangled graph homophily, which decomposes the homophily into three components: label homophily, structural homophily, and feature homophily. Label homophily encompasses previously mentioned edge homophily and node homophily, as well as other traditional homophily metrics such as class homophily (Li et al., 2023), adjusted homophily (Platonov et al., 2023a), and neighbor homophily (Gong et al., 2023). The study argues that these label-based metrics, which exclusively consider label consistency between connected nodes, offer a limited view of graph homophily. Structural and feature homophily are defined as follows.

The structural homophily h_S is calculated as:

$$h_S(\mathcal{G}, S, \boldsymbol{Y}) = \frac{1}{C}\sum_{c=1}^{C} h_{S,c},$$
$$\text{where } h_{S,c}(\mathcal{G}, S, \boldsymbol{Y}) = 1 - \frac{\sigma(\{S(u) \mid u \in \mathcal{V}, Y_u = c\})}{\sigma_{\max}}. \tag{3}$$

Here, $h_{S,c}$ denotes the structural homophily score for class c, where function $S(\cdot)$ computes the structural information (e.g., neighbor class distributions (Zheng et al., 2024)), σ denotes standard deviation, and $\sigma_{\max}$ is the maximum deviation across all classes. The structural homophily score h_S lies in the interval $[0, 1]$, where higher values reflect greater consistency in the intra-class neighbor class distribution.

For feature homophily, since the theoretical value relies on unknown latent features, we adopt an optimization method to compute an estimated value. The feature homophily score h_F averages M estimated feature-wise homophily scores $h^*_{F,m}$. The estimation process is formulated as:

$$h^*_{F,m}(\mathcal{G}, \boldsymbol{X}_{:,m}, \boldsymbol{Y}) = \arg\min_{h_{F,m}} \sum_{u,v\in\mathcal{V}, Y_u = Y_v} ||X_{u,m}(0) - X_{v,m}(0)||^2, \tag{4}$$

where the transformed feature $X_{:,m}(0)$ is defined as $X_{:,m}(0) = (I - \frac{h_{F,m}}{\rho(A)}A)X_{:,m}$, with $\rho(A)$ representing the spectral radius of the adjacency matrix A. In this calculation, the label matrix $\boldsymbol{Y}$ serves as supervisory information, constraining the summation to node pairs belonging to the same class.

The feature homophily h_F lies in $(-1, 1)$. Positive values indicate that neighboring nodes tend to share similar features (homophily), while negative values suggest distinct features (heterophily). A score of zero implies that node features are independent of the graph topology.

In summary, building upon traditional homophily metrics, disentangled graph homophily introduces structural and feature homophily to capture the structural alignment among same-class nodes and the dependence of node features on the topology. These metrics provide a more comprehensive and nuanced understanding of graph data characteristics, facilitating deeper insights into the design paradigms of graph neural networks.

4. Motivating observations

We begin with a comparative study of four real-world datasets: two heterophilic graphs (Roman-Empire and Amazon-Ratings (Platonov et al., 2023b)) and two homophilic graphs (Amazon-Photo and Amazon-Computer (Shchur et al., 2018)). Table 1 shows the three disentangled graph homophily metrics: edge homophily (as a proxy for label homophily), feature homophily, and structural homophily. As shown in Table 1, feature homophily and structural homophily vary across datasets and do not always align with label homophily, which is typically used to distinguish between homophilic and heterophilic graphs.

Table 1
Disentangled graph homophily metrics on real-world datasets.

Dataset	h_{edge}	h_{F}	h_{S}
Roman-Empire	0.0469	0.1300	0.5271
Amazon-Ratings	0.3804	1.0000	0.5256
Amazon-Photo	0.8272	0.9100	0.7559
Amazon-Computer	0.7772	1.0000	0.7628

For example, Amazon-Ratings, classified as a heterophilic graph, exhibits perfect feature homophily (1.0000), indicating strong feature–topology dependency. In contrast, Amazon-Photo, though considered homophilic, has lower feature homophily than Amazon-Ratings.

Empirical Decomposition. To further investigate the respective contributions of different information sources to node classification, we measure the class consistency of three types of features: semantic features, neighborhood labels, and structural features. For each node, we compute whether its ground-truth label matches the majority class among its corresponding neighbors in each feature space. For **semantic features**, we use the original node feature $\mathbf{x}_u$ to find its k-nearest neighbors in the feature space. The consistency is defined as:

$$\text{Consistency}_{\text{sem}}(u) = \mathbb{I}\left[y_u = \text{Majority}\left\{ y_v \mid v \in \text{kNN}(\mathbf{x}_u) \right\} \right]. \tag{5}$$

For **neighborhood labels**, we compute the majority class in the node's k-hop neighborhood $\mathcal{N}_u^{(k)}$:

$$\text{Consistency}_{\text{label}}(u) = \mathbb{I}\left[y_u = \text{Majority}\left\{ y_v \mid v \in \mathcal{N}_u^{(k)} \right\} \right]. \tag{6}$$

For **structural features**, we use the class distribution $\mathbf{d}_u^{(k)}$ over each node's k-hop neighborhood, and compute consistency via its k-nearest neighbors in this distributional space:

$$\text{Consistency}_{\text{struct}}(u) = \mathbb{I}\left[y_u = \text{Majority}\left\{ y_v \mid v \in \text{kNN}(\mathbf{d}_u^{(k)}) \right\} \right]. \tag{7}$$

These consistency scores reflect how predictive each type of feature is with respect to the true class label. We report the average value of each metric across all nodes in the dataset.

Fig. 1 shows that feature effectiveness varies significantly across datasets. Semantic features dominate in Roman-Empire, while topology-related signals are more informative in Amazon-Photo and Amazon-Computer. In Amazon-Ratings, all three sources contribute comparably. These patterns are not aligned with simple homophilic/heterophilic classification.

Insights. 1. **Binary homophilic/heterophilic classification is insufficient.** Feature importance varies significantly across datasets, even among those categorized under the same homophily regime, indicating that a coarse binary classification of graphs as homophilic or heterophilic fails to capture the nuances necessary for effective GNN design.

2. **Information utility is node-specific.** Different datasets, and even individual nodes, rely on different types of features. This highlights the need for flexible, instance-level node representations.

In summary, these findings motivate our proposed framework by highlighting the need to go beyond the homophilic/heterophilic dichotomy. They suggest that factorizing node representations into multiple feature perspectives, and dynamically adjusting their relative importance based on local graph characteristics, offers a more expressive and adaptive approach to node classification.

5. Method

To overcome the limitations of designing GNNs under the binary categorization of graphs as homophilic or heterophilic, we propose a novel

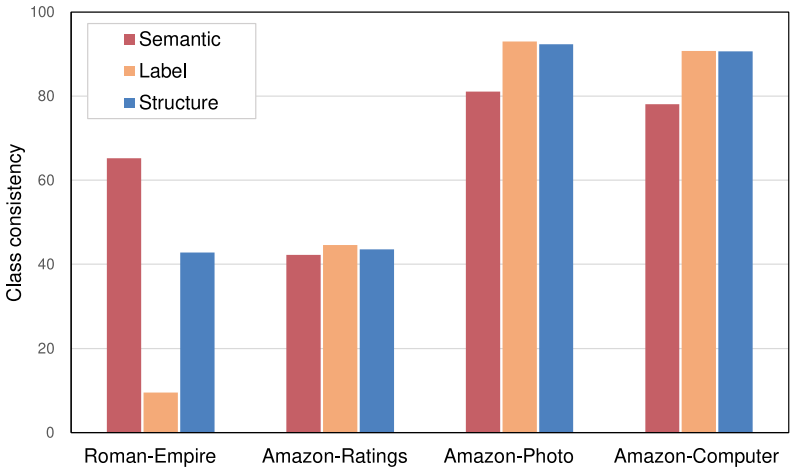


Fig. 1. Class consistency of semantic, label-context, and structural features.

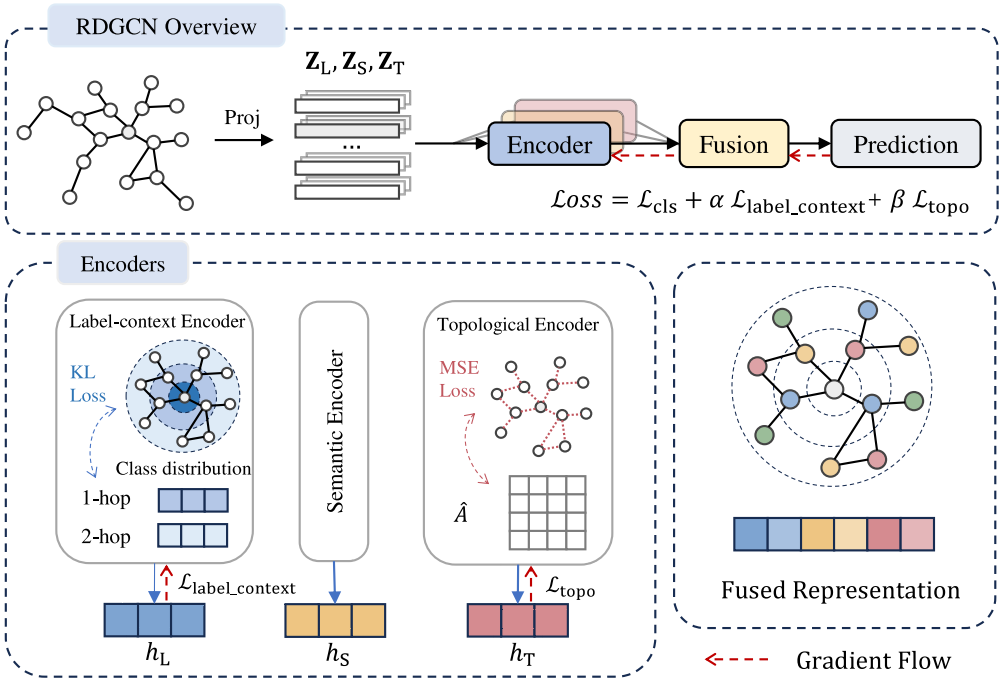


Fig. 2. An illustration of RDGCN. The core idea is to learn three view-specific representations using a label-context encoder, a semantic encoder, and a topological encoder, followed by node-level fusion. The learning of the encoders is guided by regularization losses $\mathcal{L}_{\text{label_context}}$ and L_{topo}. The final fused representation integrates view-specific semantic features, topological structure, and class-specific neighborhood awareness.

disentangled node representation. Building on this formulation, we propose a general framework named RDGCN (Representation-Disentangled Graph Convolutional Network) as illustrated in Fig. 2. RDGCN adaptively captures graph characteristics from three distinct aspects: **label context**, **semantic**, and **topological structure**, enabling a finer-grained and more interpretable node representation. In this section, we first describe how node representations are disentangled into these three perspectives, and then explain how they are integrated to support node classification.

5.1. Disentangled node representation learning

5.1.1. Rethinking node representation

We disentangle a node's representation into three components: label-context, semantic, and topological perspectives. Specifically, a node is characterized by its intrinsic semantic features and the information from its neighborhood, which is further decomposed into two complementary aspects: (1) structural connectivity to neighbors, and (2) the specific class labels of neighboring nodes. This disentanglement enables a more

comprehensive and fine-grained characterization of each node. We formalize this as follows.

Definition 1. For a given node u in a graph $\mathcal{G}$, its disentangled representation comprises three components: the label-context representation $h^{\text{label}}(u)$, which encodes the class labels of nodes in the k-hop neighborhood of u; the semantic representation $h^{\text{sem}}(u)$, which captures the intrinsic characteristics of u derived solely from its raw feature x_u; and the topological representation $h^{\text{topo}}(u)$, which reflects the connectivity structure within the k-hop neighborhood of u.

We emphasize that in this work, disentanglement does not imply strict mathematical orthogonality or statistical independence among branches. Instead, it refers to *view-specific specialization*, where different representation branches are guided by different supervisory objectives to emphasize semantic, label-context, and topological information, respectively. The overall multi-view specialization is empirically supported by branch masking analysis, node-level sensitivity, and representation similarity analyses presented in Section 7.

5.1.2. Label-context representation learning

Label-context representation learning aims to implicitly capture the class information of neighboring nodes. For a given dataset, we posit that using the node's k-hop neighborhood is sufficient to encode its label context. The k-hop neighborhood of node u contains $|\mathcal{N}_u|$ nodes with different class labels and is unordered, which necessitates that the label-context representation be permutation-invariant. We formally state the learning objective and its invariance property in Theorem 1.

Theorem 1. *Let ψ be a label-context encoder mapping a node u to a representation $\psi(u)$, and let $\mathbf{d}_u^i \in \mathbb{R}^C$ denote the class distribution over the i-hop neighborhood $\mathcal{N}_u^{(i)}$. Assume the learning objective minimizes a strictly proper discrepancy loss:*

$$\mathcal{L}_{label_context}(\psi) = \mathcal{D}\big(\psi(u), \mathbf{d}_u^i\big). \tag{8}$$

Then, the optimal encoder ψ^* that minimizes this loss is permutation-invariant with respect to the ordering of nodes in $\mathcal{N}_u^{(i)}$.

Proof. The proof proceeds in two steps. First, we observe that the target distribution $\mathbf{d}_u^i$ is derived from the *multiset* of labels within $\mathcal{N}_u^{(i)}$. Since a multiset is inherently unordered, $\mathbf{d}_u^i$ remains invariant under any permutation π of the neighborhood indices, i.e., $\mathbf{d}_u^i(\mathcal{N}_u^{(i)}) = \mathbf{d}_u^i(\pi(\mathcal{N}_u^{(i)}))$.

Second, consider the global minimum of the loss function. Since $D(\mathbf{x}, \mathbf{y}) = 0$ if and only if $\mathbf{x} = \mathbf{y}$ (assuming D is a metric or strict divergence), the optimal encoder ψ^* must satisfy $\psi^*(u) = \mathbf{d}_u^i$ for all u. Combining this with the invariance of the target, we have:

$$\psi^*(u; \pi(\mathcal{N}_u^{(i)})) = \mathbf{d}_u^i(\pi(\mathcal{N}_u^{(i)})) = \mathbf{d}_u^i(\mathcal{N}_u^{(i)}) = \psi^*(u; \mathcal{N}_u^{(i)}). \tag{9}$$

Thus, the output of the optimal encoder ψ^* is independent of the node ordering, completing the proof. □

We instantiate ψ using a k-layer GNN encoder, denoted as GNN_L, where GNN_L^i represents the i-th layer. Specifically, we train the output of each layer to predict the class distribution of the corresponding i-hop neighborhood. Prior to training, we precompute the ground-truth class distributions of the 1- to k-hop neighborhoods. In our semi-supervised setting, only labeled nodes from the training set are used for this computation. Let $\mathbf{D}^i_{\mathcal{V}_{\text{train}}} \in \mathbb{R}^{N\times C}$ denote the class distribution matrix for the i-hop neighborhoods. For nodes without labeled training neighbors within i hops, we assign an all-zero target vector, effectively excluding them from the label-context regularization term. The label-context encoder is trained via the following regularization objective:

$$\mathbf{H}_\text{L}^0 = \mathbf{Z}_\text{L} = \mathbf{X}\mathbf{W}_\text{L}, \tag{10}$$

$$\mathbf{H}_\text{L}^i = \text{GNN}_\text{L}^i(\mathbf{H}_\text{L}^{i-1}, \mathbf{A}), \tag{11}$$

$$\mathcal{L}_{\text{label_context}} = \sum_{i=1}^{k} \text{KL}\Big(\text{Softmax}(\mathbf{H}_\text{L}^i \mathbf{W}_{\text{out}}^i) \parallel \mathbf{D}^i_{\mathcal{V}_{\text{train}}}\Big), \tag{12}$$

where $\mathbf{H}_\text{L}^i$ is the hidden representation from the i-th layer, projected by a learnable matrix $\mathbf{W}_{\text{out}}^i \in \mathbb{R}^{d_k\times C}$ and normalized via softmax to obtain the predicted class distribution. The KL divergence at each layer is computed as:

$$\text{KL}(\mathbf{P}^{\text{GNN}}, \mathbf{P}^{\text{Dist}}) = \frac{1}{N}\sum_{i=1}^{N}\sum_{c=1}^{C} p_i^{\text{GNN}}(c) \log \frac{p_i^{\text{GNN}}(c)}{p_i^{\text{Dist}}(c)}, \tag{13}$$

where $p_i^{\text{GNN}}(c)$ and $p_i^{\text{Dist}}(c)$ denote the predicted and actual probabilities of class c for node i, respectively. In this way, the label-context encoder effectively "sees" the class composition of a node's k-hop neighborhood, as illustrated in Fig. 2, and contributes to the final fused representation.

5.1.3. Semantic representation learning

To capture semantic information, we first project the raw node features and then apply an MLP encoder (denoted as MLP_S):

$$\mathbf{Z}_\text{S} = \mathbf{X}\mathbf{W}_\text{S}, \tag{14}$$

$$\mathbf{H}_\text{S} = \text{MLP}_\text{S}(\mathbf{Z}_\text{S}), \tag{15}$$

where $\mathbf{W}_\text{S} \in \mathbb{R}^{d\times d_k}$ is a learnable projection matrix, and $\mathbf{H}_\text{S} \in \mathbb{R}^{N\times d_k}$ is the semantic representation. In this module, the model learns solely from the semantic features, excluding any potential interference from the graph topology or other structural information.

5.1.4. Topological representation learning

Topological representation learning aims to capture the connectivity patterns among nodes. We employ an l-layer GNN encoder, denoted as GNN_T, to generate the topological representation $\mathbf{H}_\text{T}$. To enforce topological awareness, we reconstruct the adjacency matrix using $\mathbf{H}_\text{T}$, and define a reconstruction loss. The entire process is defined as follows:

$$\mathbf{Z}_\text{T} = \mathbf{X}\mathbf{W}_\text{T}, \tag{16}$$

$$\mathbf{H}_\text{T} = \text{GNN}_\text{T}(\mathbf{Z}_\text{T}, \mathbf{A}), \tag{17}$$

$$\mathcal{L}_{\text{topo}} = \|\mathbf{A} - \sigma(\mathbf{H}_\text{T}\mathbf{H}_\text{T}^\top)\|_F^2, \tag{18}$$

where $\mathbf{W}_\text{T} \in \mathbb{R}^{d\times d_k}$, $\mathbf{Z}_\text{T}, \mathbf{H}_\text{T} \in \mathbb{R}^{N\times d_k}$. We note that since the topological branch takes projected raw features $\mathbf{Z}_\text{T}$ as input, the resulting representation $\mathbf{H}_\text{T}$ may retain some semantic information. The topology reconstruction objective is designed to bias this branch toward topology-aware patterns, rather than to guarantee strict statistical independence from the semantic branch.

Overall, our RDGCN constructs three dedicated encoders to extract label context, semantic, and topological information, respectively. Each encoder is designed to focus on a specific aspect of node representation. Furthermore, we clarify the motivation behind our encoder-specific regularization strategy. Rather than enforcing the encoders to explicitly reconstruct the neighborhood class distribution (for the label-context encoder) or the adjacency matrix (for the topology encoder), we leverage these as supervisory signals to guide the encoders toward learning implicit representations that capture the target node's neighborhood label patterns and structural characteristics.

5.2. Representation fusion and node classification

Based on the outputs of the three encoders, we obtain the label-context representation $\{\mathbf{H}_\text{L}^1, \ldots, \mathbf{H}_\text{L}^k\}$, the semantic representation $\mathbf{H}_\text{S}$, and the topological representation $\mathbf{H}_\text{T}$. Considering that nodes within the same dataset may still rely on different types of representations to varying degrees, we perform node-level fusion of the three representation types. We implement the fusion using concatenation followed by a linear transformation, which is straightforward yet sufficiently effective. We then use the fused representation $\mathbf{H}_{\text{Fused}}$ for node classification:

$$\mathbf{H}_{\text{Fused}} = \text{Linear}\big(\text{Concat}(\mathbf{H}_\text{L}^1, \ldots, \mathbf{H}_\text{L}^k, \mathbf{H}_\text{S}, \mathbf{H}_\text{T})\big), \tag{19}$$

$$\hat{\mathbf{Y}} = \text{Softmax}(\text{Linear}(\mathbf{H}_{\text{Fused}})), \tag{20}$$

Table 2
Dataset statistics.

Dataset	h_{edge}	h_{F}	h_{S}	Classes	Nodes	Edges	Features
Texas	0.0609	0.0000	0.5158	5	183	295	1703
Wisconsin	0.1778	0.0000	0.4687	5	251	466	1703
Actor	0.2167	0.0000	0.3841	5	7600	26,752	931
Squirrel	0.2072	0.0000	0.6030	5	5201	198,493	2089
Chameleon	0.2361	0.0000	0.5367	5	2277	31,421	2325
Cornell	0.1227	0.0000	0.3676	5	183	280	1703
CiteSeer	0.7391	0.7700	0.3909	6	3327	4732	3703
PubMed	0.8024	1.0000	0.3792	3	19,717	44,338	500
Cora	0.8100	0.1500	0.6164	7	2708	5429	1433

where $H_{\text{Fused}} \in \mathbb{R}^{N\times d_k}$, $\hat{\mathbf{Y}} \in \mathbb{R}^{N\times C}$. We use cross-entropy loss for training. The overall objective combines the classification loss with the two regularization losses:

$$\mathcal{L} = \mathcal{L}_{\text{cls}} + \alpha\mathcal{L}_{\text{label_context}} + \beta\mathcal{L}_{\text{topo}}, \tag{21}$$

where α and β are hyperparameters balancing the main classification objective and the regularization terms. These three components jointly ensure that node classification and encoder training are optimized simultaneously.

6. Time complexity analysis

We analyze the theoretical time complexity of RDGCN in terms of standard graph statistics: number of nodes N, number of edges E, hidden dimension d, and number of classes C. For simplicity, we focus on the dominant terms and omit encoder-specific architectural details.

The semantic encoder is implemented as a stack of feedforward layers applied independently to each node. Each layer incurs a cost of $\mathcal{O}(N \cdot d^2)$, resulting in a total complexity of $\mathcal{O}(N \cdot d^2)$.

Both label-context and topological encoders are based on standard message-passing GNNs. For sparse graphs, each layer involves a matrix multiplication and aggregation over the edge set, with complexity $\mathcal{O}(E \cdot d + N \cdot d^2)$. Since these branches process graph structure, they dominate the cost on large or dense graphs.

The fusion module aggregates the outputs of different encoders. For concatenation or attention-based fusion, the cost is typically $\mathcal{O}(N \cdot d^2)$. The final linear classifier maps each fused embedding to the output space with complexity $\mathcal{O}(N \cdot d \cdot C)$.

Summing the contributions from all modules, the overall time complexity of RDGCN is $\mathcal{O}(L(N \cdot d^2 + E \cdot d))$, where L denotes the number of layers. Importantly, this complexity is linear with respect to the number of nodes and edges, ensuring that RDGCN maintains the same scalability as standard GCNs and is efficient for large-scale graph datasets.

7. Experiments

7.1. Setup

Datasets. We evaluate RDGCN on nine benchmark datasets, including 3 homophilic graphs and 6 heterophilic graphs with edge homophily ranging from strong homophily to strongly heterophilic. As shown in Table 2, we also report the feature homophily and structural homophily scores for each dataset. The datasets include three citation networks Cora, CiteSeer, and PubMed (Namata et al., 2012; Sen et al., 2008), and two Wikipedia networks Chameleon and Squirrel (Rozemberczki et al., 2021), and an actor co-occurrence network Actor (Tang et al., 2009), and three webpage networks[1] Cornell, Wisconsin, and Texas.

Baselines. We compare RDGCN with thirteen baseline methods across four categories: (1) MLP; (2) Standard GNNs: GCN (Kipf & Welling, 2017), GAT (Veličković et al., 2018), and GraphSAGE (Hamilton et al., 2017); (3) GNNs with multi-hop information fusion: MixHop (Abu-El-Haija et al., 2019), JK-Net (Xu et al., 2018a), H2GCN (Zhu et al., 2020), GCNII (Chen et al., 2020), and Ordered GNN (Song et al., 2023); (4) GNNs with advanced aggregation strategies: GPR-GNN (Chien et al., 2021) and GGCN (Yan et al., 2022). (5) GNNs with information separation: GNN-BC (Yang et al., 2022) and M2M-GNN (Liang et al., 2024).

[1] http://www.cs.cmu.edu/afs/cs.cmu.edu/project/theo-11/www/wwkb.

Table 3
Key hyperparameter settings of RDGCN for each dataset. d_{h} is the hidden dimension; L_s, L_d, and L_t denote the numbers of layers in the semantic, label-context, and topological encoders, respectively; k is the hop count of the label-context encoder; and λ_1, λ_2 are the weights of the label-context and topological auxiliary losses. All models adopt concatenation-based fusion with a GCN backbone and the Adam optimizer.

Dataset	d_{h}	$L_s/L_d/L_t$	k	lr	λ_1	λ_2
Cora	256	1/1/2	1	1×10^{-4}	0.36	0.63
CiteSeer	256	1/2/2	2	5×10^{-5}	0.97	0.88
PubMed	64	3/2/5	2	5×10^{-3}	0.90	0.10
Chameleon	256	3/4/5	4	1×10^{-4}	0.33	0.31
Squirrel	256	3/1/2	1	1×10^{-4}	0.73	0.36
Cornell	64	2/3/1	3	5×10^{-3}	0.16	0.62
Texas	128	2/2/4	2	1×10^{-3}	0.40	0.93
Wisconsin	128	2/2/5	2	5×10^{-4}	0.98	0.52
Actor	256	2/5/1	5	1×10^{-4}	0.78	0.62

Implementation Details. The hidden dimension of RDGCN is selected from $\{64, 128, 256\}$ and the number of layers in each of the three encoders from $\{1, 2, 3, 4, 5\}$. We precompute the k-hop neighborhood class distribution, where k corresponds to the number of layers in the label-context encoder. The two auxiliary loss weights λ_1 and λ_2 are searched within $[0, 1]$. In addition, each branch has an independent dropout rate, weight decay, and normalization type (batch, layer, or ℓ_2 normalization), all tuned on the validation set. The best configuration for each dataset, obtained via hyperparameter search, is summarized in Table 3. All models are trained using the Adam optimizer for 500 epochs and evaluated over the ten public splits provided by Pei et al. (2020). All experiments are conducted on a machine equipped with an RTX 3090 GPU.

7.2. Node classification

We use accuracy as the evaluation metric for node classification. For each dataset, we train the models on the ten public splits provided by Pei et al. (2020) and report the mean and standard deviation of the test accuracy in Table 4. For baseline models, we report results from previous works (Liang et al., 2024; Song et al., 2023; Yang et al., 2022) where available; otherwise, we run the experiments ourselves. All baseline models are evaluated under the same experimental setup.

Overall, RDGCN demonstrates strong generalization ability on both homophilic and heterophilic graphs across nine datasets, achieving state-of-the-art performance on seven of them. Notably, RDGCN shows

Table 4
Experimental results on homophily and heterophily datasets. For each dataset, the boldfaced denotes the best result.

Method	Texas	Wisconsin	Actor	Squirrel	Chameleon	Cornell	CiteSeer	PubMed	Cora
MLP	80.81 ± 4.8	85.29 ± 3.3	36.53 ± 0.7	28.77 ± 1.6	46.21 ± 3.0	81.89 ± 6.4	74.02 ± 1.9	87.16 ± 0.4	75.69 ± 1.2
GCN	55.14 ± 5.2	51.76 ± 4.0	27.32 ± 1.1	53.43 ± 1.3	64.82 ± 2.2	60.54 ± 5.3	76.50 ± 1.4	88.42 ± 0.5	86.98 ± 1.3
GAT	52.16 ± 6.6	49.41 ± 4.1	27.44 ± 1.0	40.72 ± 1.6	60.26 ± 1.6	69.89 ± 5.1	76.55 ± 1.2	86.33 ± 0.5	87.30 ± 1.1
GraphSAGE	82.43 ± 6.1	81.18 ± 5.6	34.23 ± 0.99	41.61 ± 1.7	58.73 ± 1.7	75.95 ± 5.0	76.04 ± 1.3	88.45 ± 0.5	86.90 ± 1.0
MixHop	77.84 ± 7.7	75.88 ± 4.9	32.22 ± 2.3	43.80 ± 1.5	60.50 ± 2.5	73.51 ± 4.6	76.26 ± 1.3	85.31 ± 0.6	87.61 ± 0.9
JK-Net	83.78 ± 2.2	82.55 ± 4.6	35.14 ± 1.2	45.03 ± 1.7	63.79 ± 2.3	75.68 ± 6.6	76.05 ± 1.4	88.41 ± 0.5	85.96 ± 0.8
H2GCN	84.86 ± 7.2	87.65 ± 5.0	35.70 ± 1.0	36.48 ± 2.0	60.11 ± 1.9	82.70 ± 5.3	77.11 ± 1.6	89.49 ± 0.4	87.87 ± 1.2
GCNII	77.57 ± 3.8	80.39 ± 4.3	37.44 ± 1.3	38.47 ± 1.6	63.86 ± 3.0	77.86 ± 1.4	77.33 ± 1.5	90.15 ± 0.4	88.37 ± 1.3
Ordered GNN	86.22 ± 4.1	88.04 ± 1.6	37.99 ± 1.0	62.44 ± 2.0	72.28 ± 2.3	87.03 ± 4.7	77.31 ± 1.7	90.15 ± 0.4	88.37 ± 0.8
GPR-GNN	78.38 ± 4.4	82.94 ± 1.2	34.63 ± 1.2	31.61 ± 1.2	46.58 ± 1.7	80.27 ± 8.0	77.13 ± 1.7	87.54 ± 0.4	87.95 ± 1.1
GGCN	84.86 ± 4.6	86.86 ± 3.3	37.54 ± 1.6	55.17 ± 1.6	71.14 ± 1.8	85.68 ± 6.6	77.14 ± 1.5	89.15 ± 0.4	87.95 ± 1.1
GNN-BC	81.71 ± 4.3	88.11 ± 3.3	36.50 ± 1.5	61.41 ± 1.6	74.63 ± 0.9	83.66 ± 5.5	76.70 ± 0.8	88.13 ± 2.2	88.75 ± 1.2
M2M-GNN	**89.19 ± 4.5**	89.01 ± 4.1	36.72 ± 1.6	63.60 ± 1.7	75.20 ± 2.3	86.48 ± 6.1	77.20 ± 1.8	**90.35 ± 0.6**	88.12 ± 1.0
RDGCN	88.05 ± 4.4	**89.25 ± 3.1**	**38.67 ± 1.3**	**73.52 ± 0.9**	**76.71 ± 1.2**	**87.32 ± 5.2**	**77.36 ± 2.0**	88.46 ± 0.3	**89.12 ± 1.6**

significant gains on heterophilic graphs. For instance, a 9.92% improvement over M2M-GNN on Squirrel, and a 1.51% gain on Chameleon. Furthermore, RDGCN consistently outperforms GNN-BC, a model that enforces orthogonality between topology and features, across all datasets. While maintaining competitive performance on homophilic graphs like Cora and CiteSeer, RDGCN excels on heterophilic graphs, showcasing its adaptability. Moreover, RDGCN consistently outperforms all multi-hop information fusion baselines. This validates the effectiveness of our label-context regularization strategy, which implicitly encodes neighborhood labels into node representations and provides critical supplementary information for node characterization. On the highly heterophilic Texas and highly homophilic PubMed datasets, RDGCN's performance, while competitive, does not reach state-of-the-art. We conjecture that while our adaptive fusion mechanism is designed for robust generalization across diverse graphs, it may not be perfectly optimized for the unique characteristics present in these two extreme cases.

7.3. Ablation study

To better understand the contribution of each encoder and the effectiveness of our encoder regularization strategies, we conduct an ablation study by comparing RDGCN with four variants: (1) w/o Encoder$_L$: RDGCN without the label-context encoder; (2) w/o Encoder$_S$: RDGCN without the semantic encoder; (3) w/o Encoder$_T$: RDGCN without the topological encoder; (4) w/o RL: RDGCN without regularization losses for label-context and topology encoders.

We evaluate these variants on the node classification task. As shown in Table 5, RDGCN consistently outperforms all variants across datasets, validating the utility of each component. On heterophilic graphs, removing the semantic encoder results in a substantial performance drop (e.g., a 32.44% decrease on Texas), which is significantly larger than the effect of removing the label-context or topological encoders (only 0.98% and 2.68%, respectively). This suggests that raw features are more discriminative than neighborhood information in such settings. Conversely, on homophilic graphs, the impact of removing the semantic encoder is relatively minor, as neighborhood information is sufficient to guide classification. For the variant without regularization loss (RL), we observe that datasets more reliant on neighborhood information exhibit greater performance degradation (e.g., Cornell, Texas), whereas those less dependent on neighborhood signals show smaller drops (e.g., Actor). We also observe that on relatively sparse graphs such as Cornell ($|E|/|V| = 1.53$), removing the topological encoder significantly reduces performance (by 3.91%), whereas its impact diminishes in relatively dense graphs like Chameleon ($|E|/|V| = 13.80$), with only a 0.17% drop. This highlights the importance of the reconstruction loss for learning structure-aware features for graphs with lower edge connectivity.

We then compare three fusion strategies within RDGCN: (1) our default Concat fusion (RDGCN), (2) simple element-wise addition (add), and (3) attention-based fusion (attention). As shown in Table 5, the default fusion consistently achieves the best or competitive performance across all datasets.

We observe that the choice of fusion method has a significant impact on performance, particularly for datasets that are sensitive to neighborhood information, such as Texas and Cornell. The simple concatenation-based fusion clearly outperforms both element-wise addition and attention-based methods. This result highlights the potential of simple strategies, indicating that more complex mechanisms do not necessarily lead to better performance.

7.4. Visualization

To assess the contribution of each encoder to node classification, we visualize the learned node embeddings using t-SNE. We select a homophilic dataset (Cora) and a heterophilic dataset (Chameleon), and present t-SNE plots of raw features, the three representations, and their fused representation, as shown in Fig. 3. Different classes are color-coded.

On Cora, the label-context encoder exhibits the strongest influence, as its representation closely resembles the fused output. On Chameleon, the semantic encoder contributes most significantly, while the other two encoders yield less separable representations. These trends are consistent with the ablation results, where performance dropped most notably when removing the label-context encoder on Cora and the semantic encoder on Chameleon.

These results confirm that different datasets exhibit distinct inductive biases. The discriminative power of semantic, label-context, and topological features varies across datasets. RDGCN effectively captures these characteristics through its three encoders and node-wise adaptive fusion, resulting in improved classification performance.

7.5. Hyperparameter sensitivity

Fig. 4 presents the classification accuracy of RDGCN under different combinations of α and β on four representative datasets: Wisconsin, Cora, CiteSeer, and Chameleon. Both α and β are varied from 0 to 1 to explore the effect of regularization strength on model performance.

On homophilic datasets such as Cora and CiteSeer, RDGCN demonstrates strong robustness to the choice of α and β, with accuracy consistently concentrated within a narrow range (e.g., 88.3%–88.9% on Cora). For Wisconsin, although slight variation is observed, the model achieves stable performance across a broad parameter region, suggesting effective regularization behavior. In contrast, performance

Table 5
Ablation study about the three encoders, the regularization strategies, and the different fusion methods on homophily and heterophily datasets.

Method	Texas	Wisconsin	Actor	Squirrel	Chameleon	Cornell	CiteSeer	PubMed	Cora
RDGCN	**88.05 ± 4.4**	**89.25 ± 3.1**	**38.67 ± 1.3**	**73.52 ± 0.9**	**76.71 ± 1.2**	**87.32 ± 5.2**	**77.36 ± 2.0**	**88.46 ± 0.3**	**89.12 ± 1.6**
w/o Encoder$_L$	87.07 ± 4.5	86.60 ± 3.4	37.76 ± 1.1	73.32 ± 1.3	76.01 ± 0.8	82.93 ± 3.6	76.86 ± 1.8	86.73 ± 0.9	87.96 ± 1.0
w/o Encoder$_S$	55.61 ± 8.0	54.34 ± 6.0	28.67 ± 1.1	57.71 ± 2.2	67.63 ± 1.6	51.22 ± 0.0	75.81 ± 1.5	88.32 ± 0.6	88.65 ± 1.3
w/o Encoder$_T$	85.37 ± 5.3	85.66 ± 4.4	38.05 ± 0.9	72.89 ± 1.0	76.54 ± 1.2	83.41 ± 4.6	76.77 ± 1.9	88.35 ± 0.4	88.70 ± 1.1
w/o RL	85.85 ± 4.3	85.28 ± 4.0	38.17 ± 1.3	73.46 ± 1.2	76.10 ± 1.3	81.46 ± 6.6	76.77 ± 2.0	88.25 ± 0.3	88.91 ± 1.1
add	82.22 ± 5.0	87.20 ± 3.9	37.23 ± 0.8	73.05 ± 1.1	76.03 ± 1.5	81.71 ± 4.0	76.74 ± 1.7	87.25 ± 0.4	88.36 ± 0.9
attention	84.90 ± 5.9	86.66 ± 2.8	38.46 ± 1.1	72.58 ± 1.4	75.99 ± 1.1	83.41 ± 6.1	76.07 ± 1.9	87.96 ± 0.6	88.49 ± 1.1

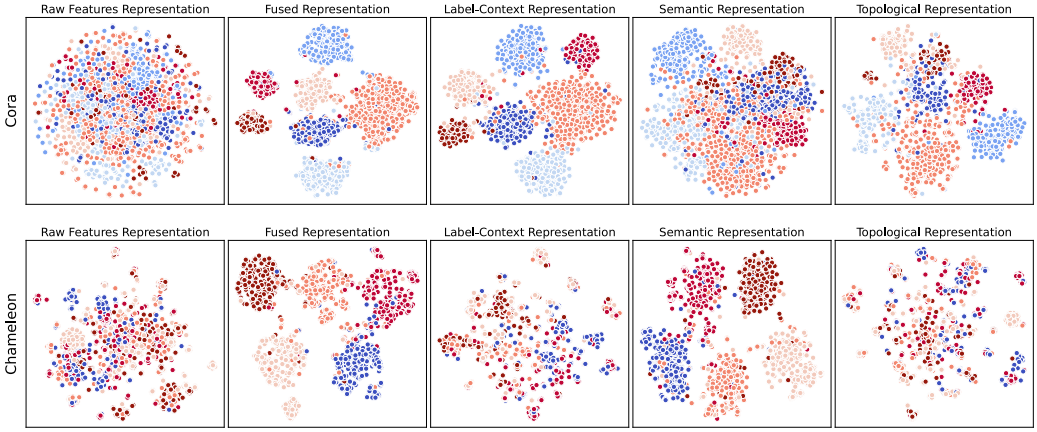
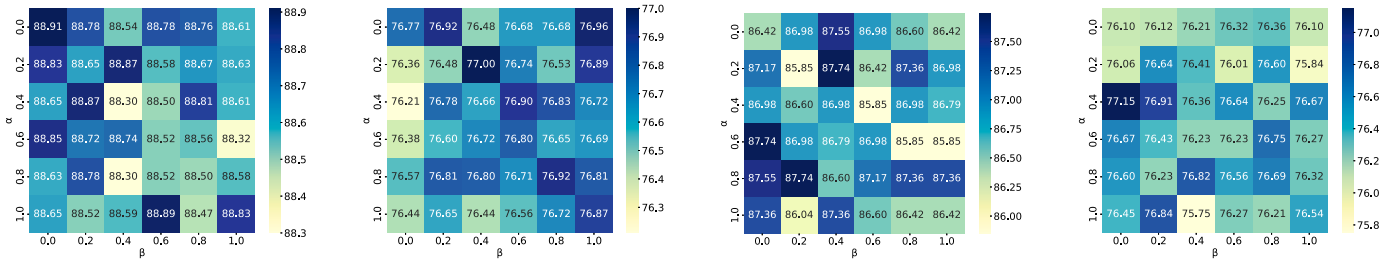


Fig. 3. t-SNE visualizations of raw features, fused representation, and the three representations (label-context, semantic, topological) on Cora and Chameleon.



(a) Cora. (b) CiteSeer. (c) Wisconsin. (d) Chameleon.

Fig. 4. Node classification accuracies as the two hyper-parameters α and β vary from 0 to 1.

on the heterophilic dataset Chameleon is more sensitive to parameter choices, with accuracy ranging from 75.75% to 77.15%. This indicates that careful tuning or adaptive strategies may be necessary when dealing with graphs with complex and diverse structural signals.

Fig. 5 shows the results of varying the number of layers in each encoder on PubMed (homophilic) and Cornell (heterophilic). We restrict the depth to 1–5 layers, without exploring deeper configurations, due to considerations of computational efficiency and memory overhead, although deeper architectures may potentially yield better performance.

On PubMed, all three encoders—semantic, label-context, and topological—yield stable performance across different depths, with marginal fluctuation (< 0.2%), confirming that node classification on homophilic graphs is less sensitive to architectural depth. In contrast, results on Cornell reveal a greater variance. Notably, the semantic encoder exhibits non-monotonic behavior, peaking at two layers, while deeper architectures cause performance degradation. Meanwhile, the label-context and topological encoders benefit from increasing depth up to a point, indicating that deeper aggregation may help capture complex neighborhood patterns in heterophilic settings.

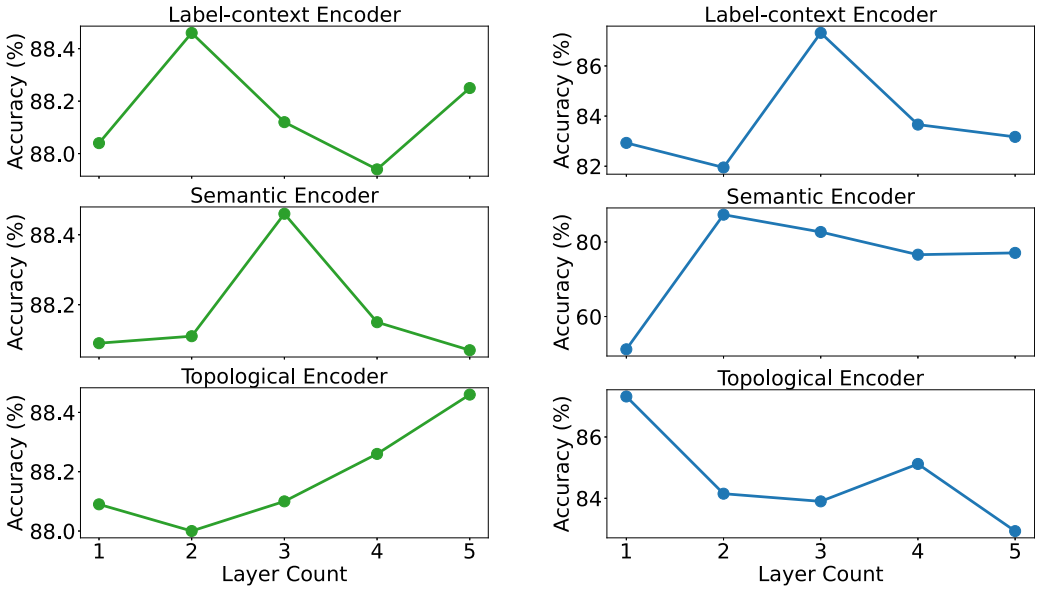
We further investigate sensitivity to hidden dimension (width). Fig. 6 reports accuracy with dimensions of 32, 64, 128, and 256.

PubMed is virtually insensitive to width, indicating that RDGCN does not uniformly require wide hidden layers. Cora and Chameleon show gradual improvements with increasing width, a common pattern in GNNs. Cornell exhibits the opposite trend: performance peaks at a moderate width and degrades with larger dimensions, likely due to overfitting on this small graph (183 nodes). Overall, RDGCN does not exhibit uniformly higher width sensitivity compared to standard methods; the effect is dataset-dependent.

7.6. Branch masking analysis

To directly assess each branch's contribution, we perform branch masking at inference time: for each branch, we zero out its representation and evaluate the resulting classification performance. Table 6 reports accuracy under four conditions: full model, mask semantic, mask label-context, and mask topological.

The results reveal that branch importance is highly dataset-dependent. On heterophilic graphs such as Texas, Wisconsin, Squirrel, Chameleon, and Cornell, masking the semantic branch causes dra-



(a) PubMed. (b) Cornell.

Fig. 5. Node classification accuracy with varying depths of label-context, semantic, and topological encoders.

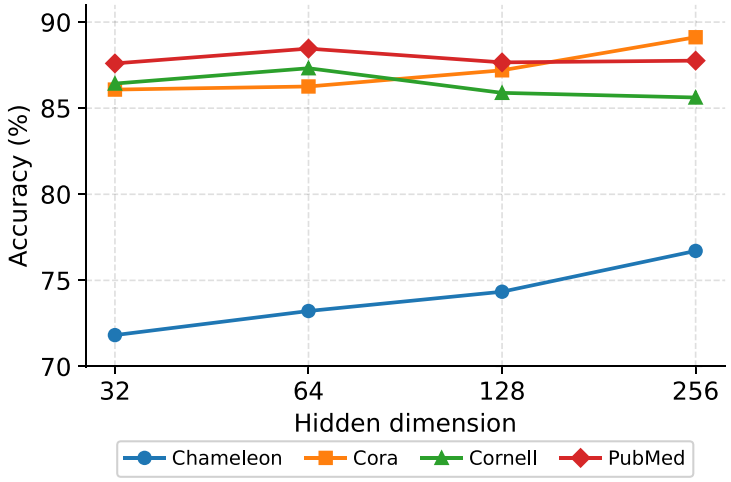


Fig. 6. Accuracy under varying hidden dimensions.

Table 6
Branch masking analysis (10 splits, mean ± std). Each row shows accuracy (%) when the specified branch is zeroed out at inference time.

Condition	Texas	Wisconsin	Actor	Squirrel	Chameleon	Cornell	CiteSeer	PubMed	Cora
Full	88.05 ± 4.4	89.25 ± 3.1	38.67 ± 1.3	73.52 ± 0.9	76.71 ± 1.2	87.32 ± 5.2	77.36 ± 2.0	88.46 ± 0.3	89.12 ± 1.6
Mask Semantic	58.38 ± 5.4	47.65 ± 10.8	23.13 ± 1.1	34.66 ± 1.7	52.68 ± 3.8	48.38 ± 20.0	75.32 ± 1.4	87.27 ± 1.3	86.62 ± 1.8
Mask Label	82.70 ± 4.3	87.45 ± 4.2	35.84 ± 1.4	69.12 ± 1.6	71.67 ± 1.5	83.24 ± 8.3	74.10 ± 1.5	34.16 ± 9.0	82.80 ± 2.6
Mask Topological	84.32 ± 4.6	87.45 ± 4.1	37.36 ± 1.0	70.39 ± 1.4	74.50 ± 1.2	83.24 ± 8.3	75.86 ± 1.5	87.71 ± 0.6	86.62 ± 1.3

matic performance drops (e.g., from 88.05% to 58.38% on Texas, and from 73.52% to 34.66% on Squirrel), while masking the label-context or topological branches has comparatively minor effects. Conversely, on PubMed, masking the label-context branch degrades accuracy from 88.46% to 34.16%, while masking the semantic branch has negligible impact. On homophilic graphs such as Cora and CiteSeer, masking the semantic branch causes only small changes, indicating that the label-context and topological branches already capture sufficient information in high-homophily settings. These results confirm that RDGCN adaptively relies on different branches depending on the intrinsic characteristics of each graph, rather than depending on a single universal dominant branch.

7.7. Node-level branch sensitivity

To empirically validate that information utility varies at the individual node level, we perform per-node branch masking. For each test node, we mask each branch independently and measure the drop in true-label probability. The branch causing the largest probability drop is designated as the node's dominant branch. Table 7 reports the proportion of test nodes dominated by each branch.

The results confirm that information utility is indeed node-specific. On Actor, all three branches account for substantial proportions of dominant nodes (56.8%, 30.1%, 13.2%), demonstrating that nodes within the same graph rely on fundamentally different information sources.

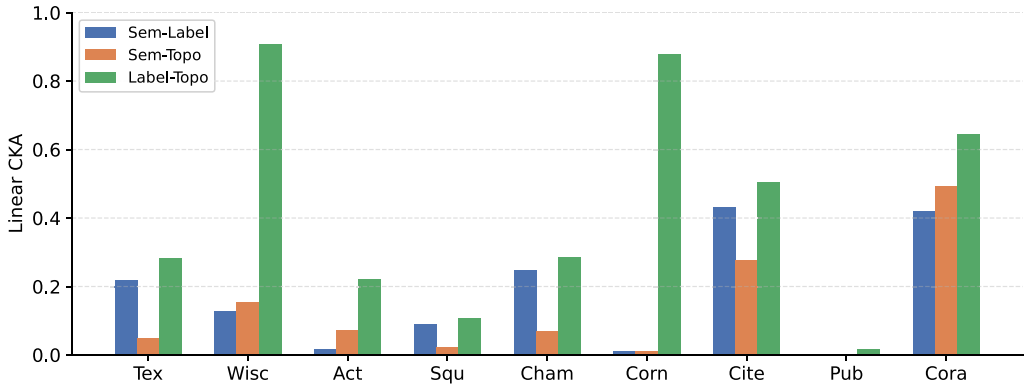


Fig. 7. Pairwise linear CKA between branch representations.

Table 7
Node-level branch sensitivity: proportion of test nodes for which each branch is dominant.

Dataset	Semantic	Label	Topological
Actor	56.8%	30.1%	13.2%
Texas	87.8%	6.5%	5.7%
Chameleon	70.4%	17.4%	12.2%
Cora	7.3%	87.4%	5.2%

Table 8
Label-context robustness under drop and noise perturbations (10 splits, mean ± std).

Dataset	Condition	Accuracy
Cora	Baseline	89.12 ± 1.6
	Drop 25%	89.00 ± 1.4
	Drop 50%	88.54 ± 1.3
	Noise 10%	88.70 ± 1.1
	Noise 30%	88.88 ± 1.2
Chameleon	Baseline	76.71 ± 1.2
	Drop 25%	76.15 ± 1.3
	Drop 50%	76.41 ± 1.1
	Noise 10%	76.50 ± 1.3
	Noise 30%	75.60 ± 1.7
Cornell	Baseline	87.32 ± 5.2
	Drop 25%	84.91 ± 7.1
	Drop 50%	86.26 ± 5.9
	Noise 10%	85.99 ± 4.6
	Noise 30%	86.80 ± 6.0

Chameleon similarly exhibits a mixed distribution, with a semantic-dominant majority but non-negligible label-context- and topology-dominant nodes. In contrast, Cora is predominantly label-context-dependent (87.4%), consistent with its high homophily, while Texas is predominantly semantic-dependent (87.8%). These findings directly support the motivation for node-level adaptive fusion in RDGCN.

7.8. Label-context robustness

To evaluate RDGCN's robustness to imperfect label-context information, we conduct perturbation experiments with two corruption types: (1) *drop*: randomly setting a fraction of label-context target vectors to zero, simulating missing label information; (2) *noise*: mixing target vectors with random one-hot distributions, simulating inaccurate labels. Table 8 reports accuracy under varying corruption levels.

Across all three datasets, RDGCN remains stable under both drop and noise perturbations at all tested corruption levels: the accuracy degradation relative to the baseline is consistently within about two percentage points and largely within one standard deviation. These results suggest that RDGCN is relatively insensitive to moderate corruption of the label-context information, indicating that the label-context branch provides complementary supervision rather than acting as a fragile dependency.

7.9. Branch representation similarity

To quantify the degree of information overlap between branches, we compute the linear Centered Kernel Alignment (CKA) between pairs of branch representations on test nodes. Fig. 7 reports pairwise CKA scores.

Across most datasets, the pairwise CKA values are low, confirming that the three branches learn largely distinct representations despite sharing the same raw feature input. In particular, the semantic branch consistently exhibits low similarity to both the label-context and topological branches. A striking case is PubMed, where all three pairwise CKA values are close to zero, indicating that its three branch representations are almost entirely dissimilar and thus strongly disentangled. We do, however, observe substantially higher label-context–topological similarity on certain graphs, most notably the homophilic Cora and the heterophilic Wisconsin and Cornell. This reflects an intrinsic correlation between local label distributions and local topological structure on these graphs. Our framework does not attempt to forcibly eliminate such correlations; instead, it uses independent encoding pathways to prevent negative interference between information sources while faithfully preserving any intrinsic correlations present in the data, leaving their integration to the fusion layer. Notably, this high label-context–topological similarity arises in both homophilic and heterophilic graphs, further supporting our central argument that categorizing graphs solely by homophily is insufficient to guide model design. Overall, these results support our formulation of view-specific specialization rather than strict independence: the branches learn complementary representations guided by different objectives, with the degree of separation depending on dataset characteristics.

8. Conclusion

In this work, we investigate the importance of different feature types for node classification and propose a novel node representation paradigm that moves beyond the binary homophilic–heterophilic view. We provide a systematic framework for factorizing node representations into view-specific components based on feature importance, enabling a unified approach that generalizes across diverse graph types. Built on this idea, RDGCN achieves strong performance using simple modules, demonstrating the potential of multi-view representation learning.

Our approach still leaves room for improvement. For instance, the topological reconstruction loss requires computing pairwise inner products, which may introduce overhead on large-scale sparse graphs. Future work may explore more efficient strategies, such as sampling-based or

localized topological reconstruction. Additionally, our branch masking analysis reveals that certain branches contribute minimally on specific datasets, suggesting that an adaptive bypass mechanism—selectively deactivating low-contribution branches during inference—could reduce computational costs without sacrificing performance. We also plan to extend RDGCN to dynamic and multimodal graphs, and investigate task-specific factorization for graph- and link-level tasks.

CRediT authorship contribution statement

Shuyang Fang: Writing – review & editing, Writing – original draft, Visualization, Validation; **Zelong Yang:** Writing – review & editing, Visualization; **Yintao Cai:** Writing – review & editing; **Yuqin Huang:** Visualization; **Xiaoping Min:** Writing – review & editing, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgement

The authors thank the anonymous reviewers for their valuable suggestions. This work was supported by the National Natural Science Foundation of China (62672426, 62272399), the Noncommunicable Chronic Diseases-National Science and Technology Major Project (2023ZD0501001), Major Science and Technology Project of Fujian Provincial Health Commission (2021ZD01006), and the Fundamental Research Funds for the Central Universities (20720250004) .

References

Abu-El-Haija, S., Perozzi, B., Kapoor, A., Alipourfard, N., Lerman, K., Harutyunyan, H., Ver Steeg, G., & Galstyan, A. (2019). Mixhop: Higher-order graph convolutional architectures via sparsified neighborhood mixing. In *International conference on machine learning* (pp. 21–29). PMLR.

Bo, D., Wang, X., Shi, C., & Shen, H. (2021). Beyond low-frequency information in graph convolutional networks. In *Proceedings of the AAAI conference on artificial intelligence* (pp. 3950–3957). (*vol. 35*).

Bruna, J., Zaremba, W., Szlam, A., & LeCun, Y. (2014). Spectral networks and deep locally connected networks on graphs. In *International conference on learning representations*.

Chen, J., Li, Z., Zhu, Y., Zhang, J., & Pu, J. (2023). From node interaction to hop interaction: New effective and scalable graph learning paradigm. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 7876–7885).

Chen, M., Wei, Z., Huang, Z., Ding, B., & Li, Y. (2020). Simple and deep graph convolutional networks. In *International conference on machine learning* (pp. 1725–1735). PMLR.

Chien, E., Peng, J., Li, P., & Milenkovic, O. (2021). Adaptive universal generalized pagerank graph neural network. In *International conference on learning representations*.

Defferrard, M., Bresson, X., & Vandergheynst, P. (2016). Convolutional neural networks on graphs with fast localized spectral filtering. In *Advances in neural information processing systems 29*.

D'Inverno, G. A., Bianchini, M., & Scarselli, F. (2025). Vc dimension of graph neural networks with pfaffian activation functions. *Neural Networks*, *182*, 106924.

Gilmer, J., Schoenholz, S. S., Riley, P. F., Vinyals, O., & Dahl, G. E. (2017). Neural message passing for quantum chemistry. In *International conference on machine learning* (pp. 1263–1272). PMLR.

Gong, S., Zhou, J., Xie, C., & Xuan, Q. (2023). Neighborhood homophily-based graph convolutional network. In *Proceedings of the 32nd ACM international conference on information and knowledge management* (pp. 3908–3912).

Guo, X., Yu, Z., Huang, F., Chen, X., Yang, D., & Wang, J. (2025). Dynamic meta-graph convolutional recurrent network for heterogeneous spatiotemporal graph forecasting. *Neural Networks*, *181*, 106805.

Hamilton, W., Ying, Z., & Leskovec, J. (2017). Inductive representation learning on large graphs. In *Advances in neural information processing systems*. (*vol. 30*).

Jin, D., Yu, Z., Huo, C., Wang, R., Wang, X., He, D., & Han, J. (2021). Universal graph convolutional networks. In *Advances in neural information processing systems* (pp. 10654–10664). (*vol. 34*).

Kipf, T. N., & Welling, M. (2017). Semi-supervised classification with graph convolutional networks. In *International conference on learning representations*. https://proceedings.neurips.cc/paper_files/paper/2024/file/7e810b2c75d69be186\penalty0cadd2fe3febeab-Paper-Conference.pdf

Klicpera, J., Bojchevski, A., & Günnemann, S. (2019). Predict then propagate: Graph neural networks meet personalized pagerank. In *International conference on learning representations*.

Li, C., & Goldwasser, D. (2019). Encoding social information with graph convolutional networks for political perspective detection in news media. In *Proceedings of the 57th annual meeting of the association for computational linguistics* (pp. 2594–2604).

Li, Q., Han, Z., & Wu, X.-M. (2018). Deeper insights into graph convolutional networks for semi-supervised learning. In *Proceedings of the AAAI conference on artificial intelligence*. (*vol. 32*).

Li, S., Kim, D., & Wang, Q. (2023). Restructuring graph for higher homophily via adaptive spectral clustering. In *Proceedings of the AAAI conference on artificial intelligence* (pp. 8622–8630). (*vol. 37*).

Liang, L., Kim, S., Shin, K., Xu, Z., Pan, S., & Qi, Y. (2024). Sign is not a remedy: Multiset-to-multiset message passing for learning on heterophilic graphs. In *International conference on machine learning* (pp. 29621–29643).

Lin, W., Chen, Z., Chen, Y., & Wang, S. (2025). Heterogeneous graph neural network with adaptive relation reconstruction. *Neural Networks*, *187*, 107313.

Liu, Z., Wan, M., Guo, S., Achan, K., & Yu, P. S. (2020). Basconv: Aggregating heterogeneous interactions for basket recommendation with graph convolutional neural network. In *Proceedings of the 2020 SIAM international conference on data mining* (pp. 64–72). SIAM.

Luan, S., Hua, C., Lu, Q., Zhu, J., Zhao, M., Zhang, S., Chang, X.-W., & Precup, D. (2022). Revisiting heterophily for graph neural networks. In *Advances in neural information processing systems* (pp. 1362–1375). (*vol. 35*).

Namata, G., London, B., Getoor, L., Huang, B., & Edu, U. (2012). Query-driven active surveying for collective classification. In *International workshop on mining and learning with graphs* (p. 1). (*vol. 8*).

Pei, H., Wei, B., Chang, K. C. C., Lei, Y., & Yang, B. (2020). Geom-GCN: Geometric graph convolutional networks. In *International conference on learning representations*.

Platonov, O., Kuznedelev, D., Babenko, A., & Prokhorenkova, L. (2023a). Characterizing graph datasets for node classification: Homophily-heterophily dichotomy and beyond. In *Advances in neural information processing systems* (pp. 523–548). (*vol. 36*).

Platonov, O., Kuznedelev, D., Diskin, M., Babenko, A., & Prokhorenkova, L. (2023b). A critical look at the evaluation of gnns under heterophily: Are we really making progress? arXiv preprint arXiv:2302.11640.

Rong, Y., Huang, W., Xu, T., & Huang, J. (2020). DropEdge: Towards deep graph convolutional networks on node classification. In *International conference on learning representations*.

Rozemberczki, B., Allen, C., & Sarkar, R. (2021). Multi-scale attributed node embedding. *Journal of Complex Networks*, *9*(2), cnab014.

Scarselli, F., Gori, M., Tsoi, A. C., Hagenbuchner, M., & Monfardini, G. (2008). The graph neural network model. *IEEE Transactions on Neural Networks*, *20*(1), 61–80.

Sen, P., Namata, G., Bilgic, M., Getoor, L., Galligher, B., & Eliassi-Rad, T. (2008). Collective classification in network data. *AI Magazine*, *29*(3), 93–93.

Shchur, O., Mumme, M., Bojchevski, A., & Günnemann, S. (2018). Pitfalls of graph neural network evaluation. arXiv preprint arXiv:1811.05868.

Song, Y., Zhou, C., Wang, X., & Lin, Z. (2023). Ordered GNN: Ordering message passing to deal with heterophily and over-smoothing. In *International conference on learning representations*.

Tang, J., Sun, J., Wang, C., & Yang, Z. (2009). Social influence analysis in large-scale networks. In *Proceedings of the 15th ACM SIGKDD international conference on knowledge discovery and data mining* (pp. 807–816).

Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., & Bengio, Y. (2018). Graph attention networks. In *International conference on learning representations*.

Wang, X., Zhu, M., Bo, D., Cui, P., Shi, C., & Pei, J. (2020). AM-GCN: Adaptive multi-channel graph convolutional networks. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining* (pp. 1243–1253).

Wu, F., Souza, A., Zhang, T., Fifty, C., Yu, T., & Weinberger, K. (2019). Simplifying graph convolutional networks. In *International conference on machine learning* (pp. 6861–6871). PMLR.

Xu, K., Hu, W., Leskovec, J., & Jegelka, S. (2019). How powerful are graph neural networks? In *International conference on learning representations*.

Xu, K., Li, C., Tian, Y., Sonobe, T., Kawarabayashi, K.-i., & Jegelka, S. (2018a). Representation learning on graphs with jumping knowledge networks. In *International conference on machine learning* (pp. 5453–5462). PMLR.

Xu, Q., Xiong, J., Cao, X., Huang, Q., & Yao, Y. (2018b). From social to individuals: A parsimonious path of multi-level models for crowdsourced preference aggregation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *41*(4), 844–856.

Yan, Y., Hashemi, M., Swersky, K., Yang, Y., & Koutra, D. (2022). Two sides of the same coin: Heterophily and oversmoothing in graph convolutional neural networks. In *International conference on data mining* (pp. 1287–1292). IEEE.

Yan, Y., Zhu, J., Duda, M., Solarz, E., Sripada, C., & Koutra, D. (2019). Groupinn: Grouping-based interpretable neural network for classification of limited, noisy brain data. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining* (pp. 772–782).

Yang, L., Zhou, W., Peng, W., Niu, B., Gu, J., Wang, C., Cao, X., & He, D. (2022). Graph neural networks beyond compromise between attribute and topology. In *Proceedings of the ACM web conference 2022* (pp. 1127–1135).

You, J., Ying, R., & Leskovec, J. (2019). Position-aware graph neural networks. In *International conference on machine learning* (pp. 7134–7143). PMLR.

Zhao, L., & Akoglu, L. (2020). PairNorm: Tackling oversmoothing in GNNs. In *International conference on learning representations*.

Zhao, T., Liu, Y., Neves, L., Woodford, O., Jiang, M., & Shah, N. (2021). Data augmentation for graph neural networks. In *Proceedings of the AAAI conference on artificial intelligence* (pp. 11015–11023). (*vol. 35*).

Zheng, Y., Luan, S., & Chen, L. (2024). What is missing for graph homophily? Disentangling graph homophily for graph neural networks. In *Advances in neural information processing systems* (pp. 68406–68452). (*vol. 37*). https://proceedings.neurips.cc/paper_files/paper/2024/file/7e810b2c75d69be186cadd2fe3febeab-Paper-Conference.pdf.

Zhou, K., Dong, Y., Lee, W. S., Hooi, B., Xu, H., & Feng, J. (2020). Understanding and resolving performance degradation in graph convolutional networks. arXiv preprint arXiv:2006.07107.

Zhu, J., Yan, Y., Zhao, L., Heimann, M., Akoglu, L., & Koutra, D. (2020). Beyond homophily in graph neural networks: Current limitations and effective designs. In *Advances in neural information processing systems* (pp. 7793–7804). (*vol. 33*).